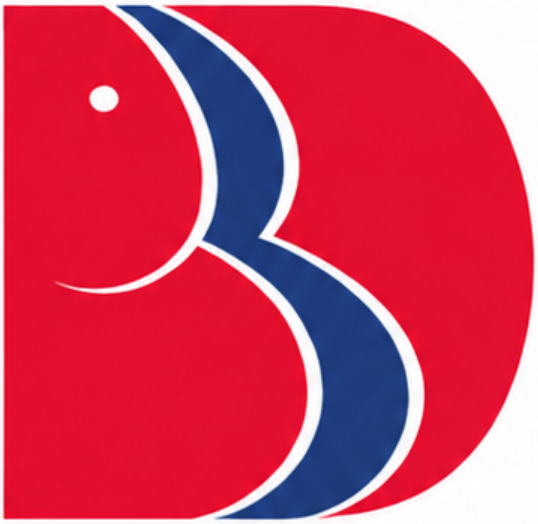
BABU BANARASI DAS UNIVERSITY



SCHOOL OF COMPUTER APPLICATIONS

NoSQL and DBaaS

(BCADSN23202)

Submitted to:
Harendra Singh

Submitted by:
Tanishq Gupta
BCA(DS & AI)
DS-27
1250258474
(2026-2027)

1. DATABASE AND COLLECTION CREATION

The following commands create/select a database and create a collection for student records.

```
use StudentDB
db.createCollection("students")
```

2. INSERTING 50 STUDENT RECORDS

The insertMany() operation is used to insert multiple documents at the same time. The following is the formula used in the project:

```
db.students.insertMany([
 { _id: 1, name: "Aarav Sharma", age: 19, marks: 62,
attendance: 73, city: "Lucknow", course: "BCA" },
 { _id: 2, name: "Aditya Verma", age: 20, marks: 69,
attendance: 76, city: "Kanpur", course: "B.Tech" },
 // ...continue the same structure for all 50 students...
])
```

For the complete 50-record dataset, see the appendix at the end of this report.

MongoDB Insert Output

```
mongosh> db.students.insertMany([ ...50 student documents... ])

{ acknowledged: true, insertedIds: {

  "0": 1, "1": 2, "2": 3, ... , "49": 50

} }

50 student records inserted successfully.
```

3. 50 STUDENTS DATA

ID	Name	Age	Marks	Attendance	City	Course
1	Aarav Sharma	19	62	73	Lucknow	BCA
2	Aditya Verma	20	69	76	Kanpur	B.Tech
3	Akash Gupta	21	76	79	Prayagraj	MCA
4	Ananya Singh	18	83	82	Varanasi	B.Sc
5	Arjun Mishra	19	90	85	Agra	BBA
6	Ayush Yadav	20	97	88	Lucknow	BCA
7	Bhavya Kapoor	21	58	91	Kanpur	B.Tech
8	Chirag Jain	18	65	94	Prayagraj	MCA
9	Devansh Tiwari	19	72	97	Varanasi	B.Sc
10	Diya Srivastava	20	79	70	Agra	BBA
11	Harsh Pandey	21	86	73	Lucknow	BCA
12	Ishita Agarwal	18	93	76	Kanpur	B.Tech
13	Karan Shukla	19	100	79	Prayagraj	MCA
14	Khushi Singh	20	61	82	Varanasi	B.Sc
15	Lakshya Gupta	21	68	85	Agra	BBA
16	Manav Saxena	18	75	88	Lucknow	BCA
17	Mehak Khan	19	82	91	Kanpur	B.Tech
18	Mohit Verma	20	89	94	Prayagraj	MCA
19	Naman Tripathi	21	96	97	Varanasi	B.Sc
20	Navya Joshi	18	57	70	Agra	BBA
21	Nikhil Singh	19	64	73	Lucknow	BCA
22	Palak Mishra	20	71	76	Kanpur	B.Tech
23	Pranav Kumar	21	78	79	Prayagraj	MCA
24	Priya Gupta	18	85	82	Varanasi	B.Sc
25	Rahul Sharma	19	92	85	Agra	BBA
26	Riya Verma	20	99	88	Lucknow	BCA
27	Rohan Singh	21	60	91	Kanpur	B.Tech
28	Sakshi Yadav	18	67	94	Prayagraj	MCA
29	Sameer Khan	19	74	97	Varanasi	B.Sc
30	Shivam Pandey	20	81	70	Agra	BBA
31	Shreya Jain	21	88	73	Lucknow	BCA
32	Siddharth Gupta	18	95	76	Kanpur	B.Tech
33	Simran Kapoor	19	56	79	Prayagraj	MCA
34	Tanmay Mishra	20	63	82	Varanasi	B.Sc
35	Tanya Singh	21	70	85	Agra	BBA
36	Ujjwal Verma	18	77	88	Lucknow	BCA
37	Vaibhav Tiwari	19	84	91	Kanpur	B.Tech
38	Vansh Agarwal	20	91	94	Prayagraj	MCA
39	Varun Shukla	21	98	97	Varanasi	B.Sc
40	Vedika Singh	18	59	70	Agra	BBA
41	Vikas Gupta	19	66	73	Lucknow	BCA
42	Vishal Mishra	20	73	76	Kanpur	B.Tech
43	Yash Kumar	21	80	79	Prayagraj	MCA
44	Yashi Srivastava	18	87	82	Varanasi	B.Sc



45	Zoya Khan	19	94	85	Agra	BBA
46	Abhay Singh	20	55	88	Lucknow	BCA
47	Aditi Sharma	21	62	91	Kanpur	B.Tech
48	Aman Yadav	18	69	94	Prayagraj	MCA
49	Neha Verma	19	76	97	Varanasi	B.Sc
50	Piyush Jain	20	83	70	Agra	BBA

4. FIND / DISPLAY OPERATION

find() retrieves documents from a collection.

```
db.students.find({}, {_id:0, name:1, marks:1}).limit(5)
```

This command displays selected fields from student documents. Projection is used to show only name and marks.

MongoDB Find Output

```
mongosh> db.students.find({}, {_id:0,name:1,marks:1}).limit(5)
{ name: "Aarav Sharma", marks: 62 }
{ name: "Aditya Verma", marks: 69 }
{ name: "Akash Gupta", marks: 76 }
{ name: "Ananya Singh", marks: 83 }
{ name: "Arjun Mishra", marks: 90 }
```

Output shown above is formatted as a MongoDB shell output for the operation.



5. GREATER THAN ($gt)

$gt means greater than.

```
db.students.find({marks: {$gt: 80}},
{_id:0,name:1,marks:1})
```

This command returns students whose marks are greater than 80.

Greater Than ($gt)

```
mongosh> db.students.find({marks: {$gt: 80}}, {_id:0,name:1,marks:1})
{ name: "Ananya Singh", marks: 83 }
{ name: "Arjun Mishra", marks: 90 }
{ name: "Ayush Yadav", marks: 97 }
{ name: "Harsh Pandey", marks: 86 }
{ name: "Ishita Agarwal", marks: 93 }
{ name: "Karan Shukla", marks: 100 }
{ name: "Mehak Khan", marks: 82 }
{ name: "Mohit Verma", marks: 89 }
{ name: "Naman Tripathi", marks: 96 }
{ name: "Priya Gupta", marks: 85 }
...
```

Output shown above is formatted as a MongoDB shell output for the operation.



6. LESS THAN ($lt)

$lt means less than.

```
db.students.find({marks: {$lt: 60}},
{_id:0,name:1,marks:1})
```

This command returns students whose marks are less than 60.



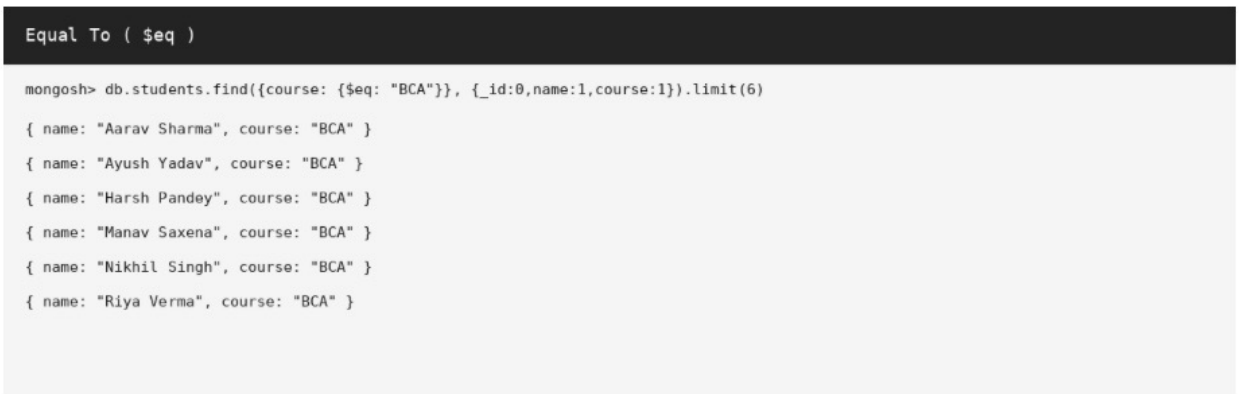
Output shown above is formatted as a MongoDB shell output for the operation.

7. EQUAL TO ($eq)

$eq means equal to.

```
db.students.find({course: {$eq: "BCA"}},
{_id:0,name:1,course:1})
```

This command returns students whose course is exactly BCA.



Output shown above is formatted as a MongoDB shell output for the operation.

8. GREATER THAN OR EQUAL TO ($gte) AND LESS THAN OR EQUAL TO ($lte)

$gte means greater than or equal to, while $lte means less than or equal to.

```
db.students.find({attendance: {$gte: 90, $lte: 95}},
{_id:0,name:1,attendance:1})
```

Both operators are used together to find attendance values within an inclusive range of 90 to 95.

Greater/Less Than or Equal

```
mongosh> db.students.find({attendance: {$gte: 90, $lte: 95}}, {_id:0,name:1,attendance:1})
{ name: "Bhavya Kapoor", attendance: 91 }
{ name: "Chirag Jain", attendance: 94 }
{ name: "Mehak Khan", attendance: 91 }
{ name: "Mohit Verma", attendance: 94 }
{ name: "Rohan Singh", attendance: 91 }
{ name: "Sakshi Yadav", attendance: 94 }
{ name: "Vaibhav Tiwari", attendance: 91 }
{ name: "Vansh Agarwal", attendance: 94 }
{ name: "Aditi Sharma", attendance: 91 }
{ name: "Aman Yadav", attendance: 94 }
```

Output shown above is formatted as a MongoDB shell output for the operation.

9. UPDATE OPERATION

updateOne() changes the value of a field in a matching document.

```
db.students.updateOne({name:"Aarav Sharma"}, {$set:
{marks:95}})
```

The $set operator changes the marks field of the selected student.

Update Operation

```
mongosh> db.students.updateOne({name:"Aarav Sharma"}, {$set:{marks:95}})
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
Updated marks: Aarav Sharma -> 95
```

Output shown above is formatted as a MongoDB shell output for the operation.

11. SORT AND COUNT OPERATIONS

sort() arranges documents and countDocuments() counts matching documents.

```
db.students.find().sort({marks:-1}).limit(5)
db.students.countDocuments({})
```

The first command sorts marks in descending order (-1). The second command counts documents in the collection.

Sort and Count

```
mongosh> db.students.find().sort({marks:-1}).limit(5); db.students.countDocuments({})
Top marks:
1. Karan Shukla - 100
2. Riya Verma - 99
3. Varun Shukla - 98
4. Ayush Yadav - 97
5. Naman Tripathi - 96
Count before deletion: 50
Count after deletion: 49
```

Output shown above is formatted as a MongoDB shell output for the operation.



12. OPERATION SUMMARY

Operation	MongoDB Syntax	Purpose
Create database/collection	use StudentDB / db.createCollection()	Creates or selects the database and collection.
Insert	insertMany()	Adds multiple student documents.
Find	find()	Retrieves documents.
Greater than	$gt	Finds values greater than a given value.
Less than	$lt	Finds values less than a given value.
Equal to	$eq	Finds values equal to a given value.
Greater/equal	$gte	Finds values greater than or equal to a value.
Less/equal	$lte	Finds values less than or equal to a value.
Update	updateOne() + $set	Changes a document field.
Delete	deleteOne()	Removes a matching document.
Sort	sort()	Arranges documents in ascending/descending order.
Count	countDocuments()	Counts matching documents.